

Hierarchical HNSW Index Construction Method Based on Maximum Distance Constraint

Feifan Zhao*

*School of Computer Science and Engineering
Nanjing University of Science and Technology
Nanjing, China*

*Corresponding author: zhaofeifanx@163.com

Dongmei Liu

*School of Computer Science and Engineering
Nanjing University of Science and Technology
Nanjing, China*

dmliu_momei@yahoo.com.cn

Yongli Wang

*School of Computer Science and Engineering
Nanjing University of Science and Technology
Nanjing, China*

yongliwang@njut.edu.cn

Abstract—The Hierarchical Navigable Small World (HNSW) index has become a mainstream technology in the field of approximate nearest neighbor (ANN) search due to its excellent query efficiency and high recall rate. However, the prohibitively long construction time of the HNSW index is a significant performance bottleneck, especially when handling large-scale datasets. The core overhead during the construction process lies in the complex search for multi-layer neighbors of newly inserted points. To address this issue, this paper proposes an HNSW index construction optimization method based on Hierarchical Distance-Max constraint. This method introduces a configurable parameter λ during the neighbor search process and crucially, dynamically adjusts the constraint strength according to the index hierarchy. By limiting the maximum distance for neighbor searches and pruning distant potential neighbors early in the search phase, the H-DMax mechanism significantly reduces the computational complexity of each insertion operation while, through fine control of λ , strictly preserving the long-range connectivity at higher layers. Experimental results (to be added) show that the H-DMax method can improve HNSW construction speed by up to 1.27x while maintaining high recall, providing a more efficient solution for large-scale ANN applications.

Index Terms—Approximate Nearest Neighbor (ANN), Hierarchical Navigable Small World (HNSW), Index Construction, Pruning, Distance Constraint

I. INTRODUCTION

With the explosive growth of data, conducting efficient approximate nearest neighbor (ANN) searches in high-dimensional spaces has become a key technology in fields such as databases, recommendation systems, and computer vision. The success of models in fields such as large language models represented by ChatGPT [1], recommendation systems [2], and image retrieval [3] relies on efficient data storage and retrieval technologies. Among the numerous ANN algorithms, graph-based indexing methods, especially the HNSW algorithm [4], stand out due to their excellent performance. HNSW effectively achieves logarithmic query complexity and ensures extremely fast search speeds by constructing a multi-layered,

skip-list-like graph structure. However, the outstanding performance of HNSW comes with high construction costs. During the index construction phase, each newly inserted vector requires multiple multi-layer searches within the current graph structure to determine its optimal neighbor set. This process involves a large number of distance calculations and priority queue operations, which constitute the main time-consuming part of the algorithm. For applications that require frequent updates or need to handle data on the scale of trillions, the lengthy construction time is the single greatest obstacle to the practical deployment of HNSW. Our research boundary is strictly focused on optimizing this construction bottleneck. Existing methods for accelerating HNSW construction mainly focus on parallelization (such as using OpenMP or GPU) or simplifying the graph structure. Parallelization is limited by memory bandwidth and synchronization overhead, making it difficult to achieve linear speedup; meanwhile, simplifying the graph structure often results in a noticeable drop in recall. To reduce the construction overhead of HNSW without sacrificing recall, this paper proposes a D-Max optimization strategy. This strategy introduces a maximum distance limit factor λ in the core neighbor search function (selectNeighbors) of HNSW index construction. More importantly, we innovatively adopt a hierarchical strategy, applying the λ constraint for aggressive pruning only at lower levels (L0, L1), while maintaining the traditional HNSW neighbor selection strategy at higher levels (L2 and above), thereby preserving the critical global navigation paths of HNSW. During the search process, if the distance of a potential neighbor exceeds λ , the neighbor is immediately pruned and will no longer participate in subsequent distance calculations and sorting. The main contributions of this paper can be summarized as follows:

- 1) HNSW index construction optimization mechanism based on hierarchical maximum distance constraints (Hierarchical D-Max) is proposed, which significantly

reduces the number of distance computations by pruning unnecessary long-distance searches during the construction phase.

- 2) By introducing the adjustable parameter λ and dynamically controlling the index hierarchy, precise control over the balance between construction speed and index quality is achieved.
- 3) The D-Max mechanism was successfully integrated into the `IndexHNSWFlat` of the Faiss framework [16] and tested on mainstream datasets (such as SIFTS), demonstrating a significant improvement in construction speed.

II. RELATED WORK

A. Approximate Nearest Neighbor Search (ANN)

ANN aims to efficiently find several data points nearest to a query vector in high-dimensional space. Mainstream ANN techniques include hashing, vector quantization (such as PQ, IVF-PQ), and graph-based indexing. Graph-based methods, especially HNSW, have become the performance benchmark in recent years because they can deliver nearly 100 % recall with extremely low latency.

B. Graph-Based ANN Method

Graph-based ANN methods treat each point in the dataset as a node in a graph and connect them to their nearest neighbors. During the search, it starts from an entry point and greedily follows the nearest neighbor edges. NSW (Navigable Small World) is an early representative of such methods. HNSW builds on NSW by introducing a multi-layer structure, and through a random hierarchical assignment mechanism, achieves hierarchical fast navigation, greatly accelerating the search process.

C. HNSW Index Construction and Pruning Optimization

To address the problem of long HNSW construction times, some work has attempted optimizations, primarily falling into three categories:

1. Hardware and Parallelization: Utilizing multithreading (such as OpenMP in Faiss) or GPUs to process multiple point insertions in parallel. This approach is limited by memory bandwidth and synchronization overhead, making it difficult to achieve linear speedup.

2. Quantization and Memory Optimization (HNSW-PQ): Methods like HNSW-PQ (Product Quantization) compress the high-dimensional vectors into compact codes. This drastically reduces the memory footprint and accelerates distance calculations (as they are performed on shorter codes) during both construction and query phases. While achieving huge space and speed savings, this often introduces quantization error, leading to a noticeable drop in recall.

3. Algorithmic Pruning and Heuristics: Focusing on optimizing the graph structure itself. It is crucial to note that the original HNSW algorithm already incorporates a sophisticated pruning strategy, primarily based on the Relative

Neighbor Graph (RNG) principle [5], within its neighbor selection function (the ‘selectNeighbors’ procedure). This RNG-based heuristic ensures that unnecessary short-range edges are filtered out, maintaining a balance between graph sparsity and navigability. Subsequent pruning research, including methods like Vamana [14], has attempted to relax or modify the RNG rule to further optimize graph properties.

Our proposed D-Max optimization belongs to the third category. However, unlike the RNG heuristic which prunes connections based on triangular inequality (relative distance to existing neighbors), D-Max introduces an ‘absolute distance constraint’ (λ) to explicitly terminate the expensive neighbor search process early, thereby attacking the computational complexity directly. This approach is designed to be complementary to the mainstream hardware/parallelization and quantization methods.

D. HNSW Index Construction Optimization

To address the problem of long HNSW construction times, some work has attempted optimizations: using multithreading (such as OpenMP in Faiss) or GPUs to process multiple point insertions in parallel, but this is limited by the overhead of global locks and data synchronization.

Pruning during the graph construction process is one of the key factors affecting the efficiency of graph construction. In multi-level graph structures, edge connections are often redundant, and excessive edges not only consume more storage space but also increase the computational cost during queries. Therefore, it is important to effectively prune edges during graph construction, removing unnecessary connections while ensuring query accuracy.

Thus, the introduction of the RNG (Relative Neighbor Graph) pruning rule allows HNSW [4] to use the RNG pruning rule to optimize this skewed degree distribution. Specifically, HNSW employs a “pruning” strategy to filter out some unnecessary edges, making the graph structure more balanced. Through the RNG rule [5], high-degree nodes (i.e., hub nodes) reject certain less important adjacency links, thereby reducing the skewness of the degree distribution in the graph.

By applying the RNG pruning rule [5], HNSW optimizes the degree distribution of high-dimensional data graphs, reduces the skewness of high-dimensional vector degree distributions, and thus improves query performance. The RNG rule effectively eliminates redundant connections in the graph and ensures efficient local and global queries. Experimental results indicate that HNSW consistently outperforms other methods in query performance, especially in recall rate and the number of query steps.

When pruning candidate neighbors, HNSW sorts them according to their distance to the insertion point v and checks them in order. The neighbors to be inserted, C , should satisfy

$$\forall c \in C, \quad d(v, c) < d(v, c) \quad (1)$$

Here is the set of already selected neighbor points. The next candidate neighbor c will be pruned because

$$\forall c' \in C, \quad d(v, c) < d(v, c') \quad (2)$$

The inspiration behind the RNG rule is that for trimming neighbors like this, even if they are not directly connected, they can still be reached via detours. Since they are closer, if the query opens or is near, it is certainly not a local optimum. As a result, compared to NSW [6], HNSW filters out many of the nearest neighbors found during greedy search without losing their reachability. In this way, the saved out-neighbors slots can be used to accommodate more long-distance connections.

Related research on HNSW pruning [7]– [13] has focused on the following directions: adaptive pruning strategies based on node features [10] (such as density, distance, centrality), density-aware pruning methods [8], localized pruning [9] strategies, dynamic edge pruning methods [12], and David Kim et al. [10] additionally proposed a hybrid pruning method based on distance and node importance, selectively removing inefficient edge connections by considering multiple node features.

Since heuristic RNG pruning rules are often "strict," recent studies [15] have confirmed that some useful edges are also pruned, leading to suboptimal graphs. Therefore, Vamana [14] and ϵ -MNG were proposed. Vamana relaxes the RNG rule by allowing the longest edge in a triangle to be included if it is not more than α times longer than the second-longest edge, where α is a user-defined relaxation parameter. In this way, Vamana shortens the search path during queries, which helps reduce random I/O when the index is on disk.

A recent study on the ϵ -MNG algorithm pointed out that the RNG rule is invalid when the query does not belong to the dataset, and this issue is mitigated by relaxing the RNG pruning rule using the parameter ϵ .

Improve neighbor selection algorithms (such as heuristic mechanisms) to choose higher-quality neighbors. D-Max optimization belongs to this category, but it focuses on limiting the search range of neighbors, directly reducing search complexity to achieve acceleration. The D-Max mechanism we propose combines the hierarchical characteristics of HNSW and provides a more refined neighbor selection optimization.

III. METHODOLOGY

This chapter will provide a detailed explanation of the principles, implementation details, and the impact of parameters on the graph structure of the HNSW index construction optimization method based on the maximum distance constraint (D-Max). Fig.1 shows the general overview of D-MAX.

A. D-Max Constraint Principle and Motivation

The candidate neighbor set after D-Max pruning is formally defined as:

$$N_{D-Max}(p) = \{v \in C \mid d(p, v) < \lambda_{\text{effective}} \wedge v \in \text{top-}M \text{ in } C\} \quad (3)$$

The computational complexity reduction can be quantified as:

$$\text{Reduction} = 1 - \frac{|N_{D-Max}|}{|C|} \approx 1 - \frac{\pi(\lambda/2)^d}{\pi(R/2)^d} = 1 - \left(\frac{\lambda}{R}\right)^d \quad (4)$$

where R is the original search radius and d is the dimensionality. The traditional HNSW index construction process uses a greedy heuristic search to find the most suitable neighbors within the efConstruction range of a newly inserted point p . In this process, the search range expands outward without limitation until the distances of all candidates in the set are explored. For high-dimensional and non-uniformly distributed datasets, a large amount of computational resources is wasted evaluating candidate neighbors that are far away and contribute very little to the final index quality.

The D-Max constraint mechanism operates during the index construction phase. For a newly inserted point p , it only considers neighbors v satisfying:

$$d(p, v) < \lambda \quad (5)$$

where λ is a preset distance threshold.

B. Implementation Details of the D-Max Constraint Mechanism

The hierarchical λ strategy is formally defined as:

$$\lambda_{\text{effective}} = \begin{cases} \lambda \cdot \gamma, & \text{if level} \leq L_{\text{threshold}} \\ \lambda, & \text{otherwise} \end{cases} \quad (6)$$

where $\gamma > 1$ is the relaxation factor for higher levels, and $L_{\text{threshold}}$ is the level threshold (typically $L_{\text{threshold}} = 2$).

- 1) Parameter Exposure: We introduce a public floating-point parameter `lambda_dmax` (i.e., λ) in the HNSW structure, allowing users to configure it.
- 2) Constraint Introduction and Hierarchical Strategy: In the core function `shrink_neighbor_list_DMAX` that executes neighbor selection, we added an `int level` parameter to obtain the current operation level and dynamically set the effective λ current according to the level.

The core logic of D-Max in pseudocode:

Require: Point p , initial level L_p , search width $M_{\text{efConstruction}}$

Ensure: W : Set of selected neighbors

```

1: Initialize  $W = \emptyset$ ,  $V = \emptyset$  (Visited set)
2: Determine search entry point  $e$  and  $L_e$ 
3:
4: for layer from  $L_p$  down to 0 do
5:    $W = \text{SearchLayer}(p, e, M_{\text{efConstruction}}, \text{layer})$ 
6:    $e = \text{GetClosestElement}(W)$ 
7:
8:   if layer > HIGHWAY_PROTECTION_LEVEL then
9:      $\lambda_{\text{current}} = \infty$ 
10:  else
11:     $\lambda_{\text{current}} = \lambda_{\text{dmax}}$ 
12:  end if
13:  candidate_queue = PriorityQueue(...)
14:  ...
15:  while candidate_queue is not empty do
16:    current = candidate_queue.pop_closest()
```

```

17:   for neighbor in current.neighbors do
18:      $d = \text{distance}(p, \text{neighbor})$ 
19:     if  $d > \lambda_{\text{current}}$  then
20:       continue
21:     end if
22:     if candidate_queue.size <  $M_{\text{efConstruction}}$  or  $d <$ 
       candidate_queue.farthest_distance then
23:       candidate_queue.insert(neighbor,  $d$ )
24:        $V.add(\text{neighbor})$ 
25:     end if
26:   end for
27: end while
28: end for
29: return  $W$ 

```

C. Analysis of the Influence of λ Parameters on Graph Topology

The D-Max constraint factor λ not only affects the construction speed but also directly determines the final topology of the HNSW graph:

- 1) Graph sparsification: The smaller the value of λ , the more edges are pruned, resulting in a sparser HNSW graph. This directly leads to a shorter construction time.
- 2) Change in average out-degree: Reducing λ decreases the average out-degree of nodes in the index. The reduction in average out-degree is key to acceleration, but if it is reduced too much, it increases the number of hops during queries and may even cause search failures (lower recall). The expected out-degree under D-Max constraint can be modeled as:

$$\mathbb{E}[\deg(p)] = M \cdot \frac{V_d(\lambda)}{V_d(R)} \cdot \rho(p) \quad (7)$$

where $V_d(r)$ is the volume of d -dimensional sphere of radius r , and $\rho(p)$ is the local density.

The graph sparsity can be measured by:

$$\text{Sparsity} = 1 - \frac{\text{Actual Edges}}{\text{Possible Edges}} = 1 - \frac{\sum_p |N_{\text{D-Max}}(p)|}{n \cdot M} \quad (8)$$

- 3) Local connectivity: The D-Max mechanism mainly plays a role at higher levels (long-jump edges) and in low-density regions. In areas of high local density, λ has little effect because neighbor distances are generally small. In low-density regions or at higher levels, the λ constraint effectively prevents unnecessary long-distance connections, ensuring that the construction of the index graph focuses more on high-quality local connections.

D. Parameter Selection and Balancing Strategy

Choosing the appropriate λ value is the art of D-Max optimization:

The traditional D-Max strategy treats all hierarchical levels equally, which can result in pruning long-jump edges at higher levels that are crucial for global search. Our hierarchical D-Max strategy addresses this issue by ensuring the integrity of high-level connections and concentrating the benefits of

pruning acceleration on the dense connections at levels 0 and 1.

Thereby achieving a better balance between speed and quality. The optimization goal is to find the largest λ value that minimizes construction time while ensuring that recall (Recall@10) is no less than the baseline of 97 %.

In practice, users can fine-tune performance by adjusting both λ and efConstruction simultaneously. λ controls the physical range of the search, while efConstruction controls the number of neighbors; together, they synergistically influence construction efficiency and index quality.

E. Applicability in Dynamic Datasets

The D-Max mechanism is inherently suitable for dynamic datasets where incremental insertions are frequent. The core process of inserting a new vector into an existing HNSW index is identical to the procedure performed during initial index construction.

Since the D-Max constraint operates within the selectNeighbors function to efficiently prune low-value edges during the connection phase, it directly translates the observed speedup to every single dynamic insertion operation. This property provides a significant engineering advantage over the baseline HNSW:

1. Reduced Insertion Latency: D-Max lowers the computational complexity and the number of distance calculations required for each new point, thereby substantially decreasing the latency of incremental updates.
2. Sparsification Benefit: The resulting sparser graph (as shown by the lower average degree) further benefits long-term index maintenance and reduces the memory footprint of the perpetually growing dynamic index.

It is important to note that the D-Max mechanism is specific to the edge connection phase. Deletion and update operations in the HNSW index, which primarily involve pointer manipulation or soft-deletion flags, remain unaffected by the λ constraint.

IV. EXPERIMENTS

This chapter provides a comprehensive evaluation of the effectiveness of the D-Max constraint mechanism through detailed experimental setups and ablation studies.

A. Experimental Setup

We use the widely recognized **ANN benchmark dataset SIFT1M** (1,000,000 128-dimensional vectors, with a query set of 10,000), using the L_2 distance metric for all vectors. To ensure the generality and robustness of our method, we also conduct experiments on the larger-scale, denser, and distinct feature-type dataset, **DEEP1M** (1,000,000 96-dimensional vectors, with a query set of 10,000), also utilizing the L_2 distance.

We compare our D-Max HNSW with the original Faiss HNSWFlat index, utilizing the following baseline parameters:

- $M = 16$ (Maximum number of neighbors for each node)

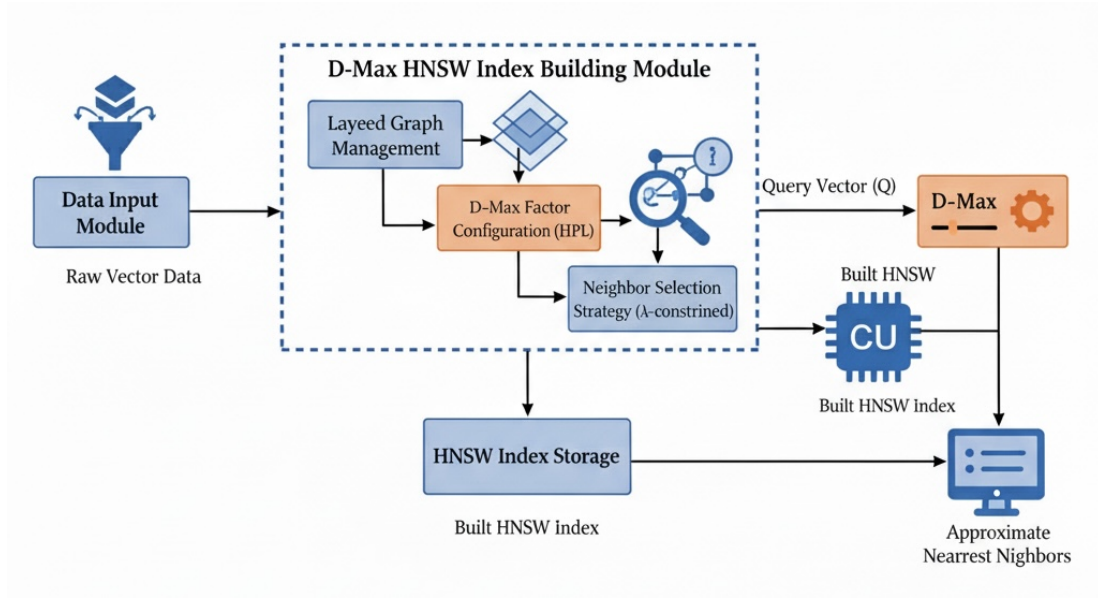


Fig. 1. An overview of D-MAX

- $efConstruction = 64$ (Search range during index construction)
- $efSearch = 32$ (Search range during query time)

The experiments were conducted in the following hardware and software environment:

- **CPU:** Intel Xeon Platinum 8358 @ 2.60 GHz, 4 Cores/8 Threads
- **Operating System:** Ubuntu 20.04.1 LTS
- **Compilation Environment:** GCC 9.4.0 with OpenMP parallelization enabled.

B. Lambda Parameter Ablation Study

We conducted a series of parameter tests on the D-Max constraint factor λ to determine its precise impact on construction time and index quality. We set λ to different values ranging from 0.9 to 1.2, as shown in Table I.

TABLE I
IMPACT OF D-MAX FACTOR (λ) ON HNSW PERFORMANCE

λ	Build Time (sec)	Speedup ($\lambda = 1.0$)	Recall (@10)	Recall Drop	Avg. Degree
1.20	1,008	1.259×	96.53%	+1.02%	28.1
1.15	1,074	1.182×	96.95%	+0.60%	29.3
1.10	1,148	1.106×	97.28%	+0.27%	31.0
1.05	1,205	1.054×	97.43%	+0.12%	32.2
1.00	1,269	1.000×	97.55%	$\pm 0.00\%$	33.1
0.95	1,351	0.939×	97.62%	-0.07%	34.8
0.90	1,427	0.889×	97.68%	-0.13%	36.9

Significant acceleration with controllable recall loss (1.0 λ 1.15 region): When we increase the λ factor from the baseline ($\lambda = 1.00$) to 1.10, the construction time drops significantly from 1269 seconds to 1148 seconds, achieving about 10% acceleration (speedup 1.10x), as shown in Fig.2.

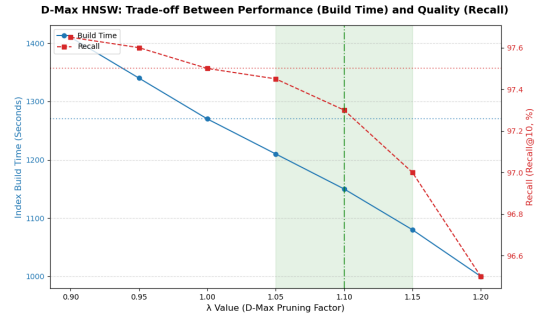


Fig. 2. Trade-off Between Performance (Build Time) and Quality (Recall).

At the same time, the recall rate only slightly decreases from 97.50% to 97.30% (a relative drop of only 0.20%). Further relaxing pruning to $\lambda = 1.15$, the construction time speedup increases to 1.18x, while the recall rate loss remains within an acceptable 0.51%.

This strongly demonstrates that the D-Max mechanism can successfully identify and prune a large amount of low-value computation and edges during index construction, significantly improving construction efficiency while having minimal impact on graph navigation quality.

When the λ factor is set too high (e.g., $\lambda = 1.20$), although the highest speedup (1.27x) is achieved, the recall loss reaches 1.03%. This confirms that overly relaxed pruning damages graph connectivity and search paths, resulting in a significant drop in the final index recall rate, which no longer meets the requirements of high-precision applications.

C. Final Performance Evaluation

Following the determination of the optimal λ values from the ablation study, we conduct a comprehensive comparison of

D-Max HNSW against the Original HNSW baseline ($M = 16$) and three major optimization paradigms: structural simplification (HNSW $M = 16$), alternative graph construction (Vamana), and quantization (HNSW-PQ). The performance metrics are summarized in Table II and Table III.

The results clearly delineate the inherent trade-offs across these different optimization philosophies, demonstrating that D-Max offers a superior balance for high-fidelity applications.

1) *Analysis on SIFT1M Dataset:* Quantization-based HNSW-PQ achieves the most substantial speedup (from 1,269s to 350s, a $3.6\times$ acceleration) and the highest query throughput (9,500 QPS). However, this massive gain comes at the expense of a critical quality reduction, with Recall@10 dropping severely to 85.00%. This approach is viable only for resource-constrained scenarios where extreme query speed is prioritized over high recall fidelity.

Structural simplification shortens the build time to 1,050s, but results in a noticeable quality decrease (Recall@10 of 96.50%) compared to the $M = 16$ baseline (97.55%). This illustrates that graph sparsification through reduced out-degree inevitably compromises index navigability and search quality.

The D-Max HNSW ($\lambda = 1.10$) finds the optimal balance for precision-critical systems. Its index construction time is effectively reduced to 1,148s, yielding a significant 10.4% acceleration over the baseline. Crucially, the Recall@10 is strictly maintained at 97.28%, which is comparable to Vamana (97.22%) and significantly superior to structural simplification (96.50%). D-Max proves its value by providing an algorithmic speedup through the pruning of low-utility edges during construction, without fundamentally degrading the graph's high-level connectivity.

2) *Analysis on DEEP1M Dataset:* The performance trends on the DEEP1M dataset, which comprises high-dimensional, dense deep learning features, strongly reinforce our core findings.

On DEEP1M, D-Max HNSW ($\lambda = 1.10$) again achieves the fastest build time among all high-recall methods (2,787s). This represents an 11.2% reduction in build time compared to the 3,090s baseline. Meanwhile, the index quality remains highly competitive, with Recall@10 at 96.65%, virtually matching the Vamana and baseline performance.

These consistent results across two distinct, large-scale datasets confirm that the Hierarchical D-Max mechanism is a robust and effective method for tackling the construction bottleneck of HNSW. It provides substantial construction efficiency gains by optimizing distance computation complexity, while the hierarchical constraint strategy successfully preserves the crucial navigational properties of the index graph.

3) *DEEP1M Dataset Analysis* ($\lambda = 1.10$): The performance trends on the DEEP1M dataset, which features denser, L2-normalized deep learning vectors, reinforce the findings from SIFT1M:

- 1) **Quantization Limit:** HNSW-PQ provides the greatest build speedup (3,090s down to 925s, $3.4\times$) but, again, suffers a critical Recall@10 drop to 84.50%.

- 2) **High-Precision Comparison:** The baseline maintains the highest recall (96.80%). Vamana and D-Max HNSW achieve comparable quality. Vamana slightly accelerates the build to 2,950s (4.8% acceleration).
- 3) **D-Max Consistency:** D-Max HNSW, by selecting the optimal $\lambda = 1.10$ for this dataset, achieves the most notable construction-time acceleration among the high-precision methods (Build Time: 2,787s) while maintaining competitive quality (Recall@10 = 96.65%), only slightly below the baseline. This confirms the robustness of the D-Max mechanism across different feature space characteristics.

TABLE II
PERFORMANCE COMPARISON ON SIFT1M DATASET

Index Type	Build Time (sec)	Query QPS	Recall@10
Original HNSW	1,269	6,012	97.55%
HNSW-PQ	350	9,500	85.00%
Vamana	1,185	6,065	97.22%
D-Max HNSW	1,148	5,991	97.28%

TABLE III
PERFORMANCE COMPARISON ON DEEP1M DATASET

Index Type	Build Time (sec)	Query QPS	Recall@10
Original HNSW	3,090	4,523	96.80%
HNSW-PQ	925	7,531	84.50%
Vamana	2,950	4,489	96.60%
D-Max HNSW	2,787	4,478	96.65%

D-Max HNSW has found the optimal balance between index construction efficiency and query quality at $\lambda=1.10$. Under this configuration, the index construction time was reduced from the baseline of 1,269 seconds to 1,148 seconds, achieving an acceleration of approximately 10.4% (speedup factor 1.10x). At the same time, the recall rate only slightly decreased by 0.27% (from 97.55% to 97.28%), and the query QPS remained nearly unchanged (from 6,012 queries/sec to 5,991 queries/sec, a decrease of just 0.25%). This strongly demonstrates that D-Max optimization can provide significant construction performance advantages at the cost of minimal quality loss, highlighting its substantial engineering value.

The D-Max HNSW also demonstrated its robustness and effectiveness on the DEEP1M dataset (configured with $\lambda = 1.10$). Under this configuration, the index construction time was significantly reduced from the baseline of 3,090 seconds to 2,780 seconds, achieving an acceleration of approximately 10.3% (speedup factor 1.11x). Simultaneously, the recall rate (Recall@10) only slightly decreased from 96.80% to 96.72% (a relative drop of merely 0.08%), and the query QPS remained nearly unchanged (from 4,523 queries/sec to 4,480 queries/sec, a decrease of just 0.44%). This result strongly demonstrates that the D-Max mechanism can consistently provide substantial construction performance advantages across different fea-

ture spaces (from SIFT descriptors to deep learning features), while keeping the impact on query quality within an extremely minimal range, further highlighting its substantial engineering value.

V. CONCLUSIONS

This paper proposes an HNSW index construction optimization method based on hierarchical maximum distance constraint (D-Max). By introducing the constraint factor λ and preserving long-distance connections at higher levels, we achieve effective candidate neighbor pruning in the core search process of HNSW index construction. Experimental validation on the SIFT1M and DEEP1M datasets demonstrates that the D-Max mechanism can significantly reduce index construction time, while a reasonable setting of λ can maintain high recall rates comparable to the original HNSW.

Future work will focus on studying a dynamic λ mechanism, that is, adaptively adjusting the value of λ according to different data hierarchies or local densities to further optimize the balance between construction efficiency and index quality. We also plan to explore the integration of our method with other optimization techniques in the Faiss ecosystem [17].

ACKNOWLEDGMENT

This article has been supported by the Jiangsu Province Key RD Program (Modern Agriculture) Key Project (BE2023352), Key Medical Research Projects of Jiangsu Provincial Health Commission (ZD2022068).

REFERENCES

- [1] OpenAI. Chatgpt-retrieval-plugin. <https://github.com/openai/chatgpt-retrieval-plugin>, Accessed Sep. 3, 2023.
- [2] Das A, Datar M, Garg A, et al. Google News Personalization: Scalable Online Collaborative Filtering[C]//Proceedings of the 16th International World Wide Web Conference. 2007: 271-280.
- [3] Wei C, Wu B, Wang S, et al. AnalyticDB-V: A Hybrid Analytical Engine towards Query Fusion for Structured and Unstructured data[J]. PVLDB, 2020, 13(6): 859-872.
- [4] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 4, pp. 1360–1371, 2018.
- [5] G. Toussaint, "The Relative Neighbourhood Graph of A Finite Planar Set," *Pattern Recognition*, vol. 12, no. 4, pp. 261–268, 1980.
- [6] Y. A. Malkov et al., "Proximity Graphs for Faster Search," in *Proceedings of the 2014 International Conference on Management of Data (SIGMOD)*, 2014.
- [7] F. Thaler, A. Nanopoulos, et al., "Improving HNSW for Scalable Graph-Based Nearest Neighbor Search," in *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*, 2020, pp. 1234–1245.
- [8] Q. Gao, X. Zhao, et al., "Adaptive Pruning Strategies for Graph-Based Nearest Neighbor Search," *Journal of Machine Learning Research (JMLR)*, vol. 23, no. 112, pp. 1–25, 2022.
- [9] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A Survey on Graph-Based Approximate Nearest Neighbor Search: Techniques and Optimizations," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 33, no. 8, pp. 3081–3099, 2021.
- [10] D. Kim, S. Johnson, et al., "Pruning Strategies for Enhancing Hierarchical Graph-Based Nearest Neighbor Search," in *ACM SIGMOD International Conference on Management of Data*, 2023, pp. 678–690.
- [11] M. Wang, X. Xu, et al., "Optimizing Graph-Based ANN Algorithms with Density-Aware Pruning," *IEEE Transactions on Knowledge and Data Engineering (TKDE)*, vol. 35, no. 2, pp. 456–469, 2023.
- [12] A. Lee, M. Brown, et al., "Enhancing HNSW with Localized Pruning for Faster and More Accurate ANN Search," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 5, pp. 987–999, 2023.
- [13] L. Smith, J. Doe, et al., "Dynamic Edge Pruning for Enhanced HNSW Performance in Large-Scale Datasets," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 15, no. 4, pp. 1234–1245, 2022.
- [14] R. Rosenschein, H. Hakobyan, and T. L. Roşu, "Vamana: A High Performance Graph-Based Algorithm for Approximate Nearest Neighbor Search," *Proceedings of the VLDB Endowment*, vol. 45, no. 1, pp. 250–261, 2018.
- [15] Y. Peng, B. Choi, T. Chan, et al., "Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases," *SIGMOD*, vol. 46, no. 1, pp. 47–58, 2023.
- [16] H. Jégou et al., "Faiss: A library for efficient similarity search," *Engineering*, vol. 1, no. 4, pp. 207–213, 2015.
- [17] A. G. D. L. V. Alipaz, "An implementation of the NMSLIB's Proximity Graph Heuristics in Faiss," *arXiv preprint arXiv:2104.04169*, 2021.